BXP System Architecture

Author: Elvarin
Version: BXP v2.0 / Reference Server v2.1
Date: 2026

Overview

This document describes the architecture of the Breathe Exposure Protocol (BXP) at four levels:

1. High-level ecosystem
2. Data flow through a BXP node
3. Interoperability model
4. Data lifecycle

For each diagram, the implementation status is noted: whether the component is built, specified, or proposed.

Architecture 1 — High-Level BXP Ecosystem

What this represents: The full BXP ecosystem as designed, showing all actors, components, and how they relate. Components marked with [proposed] describe the intended long-term architecture; [implemented] describes what is built and working in this repository.

```
graph TB
    subgraph Sources["Data Sources (any of these)"]
        S1[" Phone / App\n(Tier 1)"]
        S2[" Consumer Sensor\n(Tier 2)"]
        S3[" Government Station\n(Tier 3)"]
        S4[" Satellite Data\n(Tier 2)"]
        S5[" Community Observer\n(human report)"]
    end

    subgraph SDK["BXP SDK [IMPLEMENTED]"]
        PY["Python SDK\nbxp_sdk.py"]
        TS["TypeScript SDK\nbxp-sdk.ts"]
        CLI["CLI Tool\nbxp_cli.py"]
        MQTT["MQTT Bridge\nmqtt_bridge.py"]
    end

    subgraph Node["BXP Reference Node [IMPLEMENTED]"]
        API["FastAPI REST Server"]
        DB["SQLite Database"]
        HRI["HRI Calculator"]
        QC["QC Pipeline"]
        PRIV["Privacy Layer\n(geohash floor, k-anon)"]
        DASH["Web Dashboard\n(map + charts)"]
    end

    subgraph FedNet["Federated Network [PROPOSED]"]
```

```
        N1["Community Node"]
        N2["Research Node"]
        N3["Government Node"]
        REG["Node Registry"]
    end

    subgraph Apps["Applications"]
        APP1["Air quality app"]
        APP2["Research dashboard"]
        APP3["Government portal"]
        APP4["Embeddable widget"]
    end

    S1 & S2 & S3 & S4 --> SDK
    S5 --> Node
    SDK --> Node
    Node --> FedNet
    FedNet --> Apps
    Node --> Apps
    AQICN["AQICN API\n(live city data)"] --> Node
```

Architecture 2 — Data Flow Through a BXP Node

What this represents: How a single reading flows from submission to storage to retrieval inside the reference server. All components shown here are implemented and working.

```
sequenceDiagram
    participant Client as Client (SDK / curl / app)
    participant API as FastAPI Server
    participant RL as Rate Limiter
    participant AUTH as Auth (Device Token)
    participant QC as QC Pipeline
    participant HRI as HRI Calculator
    participant PRIV as Privacy Layer
    participant DB as SQLite Database
    participant DASH as Dashboard

    Client->>API: POST /bxp/v2/readings
    API->>RL: Check rate limit (IP)
    RL-->>API: Allowed / 429
    API->>AUTH: Validate Bearer token (optional)
    AUTH-->>API: Device UUID / anonymous

    loop For each reading
        API->>QC: Validate fields, range-check lat/lon/values
        QC-->>API: quality flag + confidence
        API->>HRI: Calculate BXP_HRI(agents, duration, population)
        HRI-->>API: score, level, color, advice
        API->>PRIV: Apply geohash floor (precision≥5)
        PRIV-->>API: stored_geohash
        API->>API: Compute SHA-256 payloadHash
        API->>DB: INSERT reading record
    end

    API-->>Client: 201 {readingId, bxpHri, bxpHriLevel, qualityFlag}
```

```
    Client->>API: GET /bxp/v2/locations/s1v0g/latest
    API->>DB: SELECT latest for geohash
    DB->>API: record
    API->>Client: 200 {reading, bxpHri}

    DASH->>API: Auto-refresh (60s polling)
    API->>DB: Query latest readings
    API->>DASH: Updated map + chart data
```

Architecture 3 — Interoperability Architecture

What this represents: How BXP achieves interoperability — any conforming source can write data any conforming application can read. The key insight is that BXP is a protocol layer that sits between sources and applications, eliminating the N×M integration problem.

```
graph LR
    subgraph Without BXP
        direction TB
        WS1["Source A\n(proprietary format)"] --> WA1["App 1\n(custom parser A)"]
        WS1 --> WA2["App 2\n(custom parser A)"]
        WS2["Source B\n(different format)"] --> WA1
        WS2 --> WA2
        WS3["Source C\n(yet another format)"] --> WA1
        WS3 --> WA2
    end

    subgraph With BXP
        direction TB
        BS1["Source A"] --> BXP[".bxp.json\nUniversal Schema"]
        BS2["Source B"] --> BXP
        BS3["Source C"] --> BXP
        BXP --> BA1["App 1"]
        BXP --> BA2["App 2"]
        BXP --> BA3["App 3"]
    end
```

The N×M problem: Without a standard, N sources and M applications require N×M custom integrations. With BXP, N sources need N integrations to write BXP, and M applications need M integrations to read BXP — total N+M.

What makes BXP-conformant implementations interoperable:

```
graph TD
    A["Universal Agent IDs\n(PM2_5, NO2, O3...)"] --> I["Interoperability"]
    B["Canonical Units\n(µg/m³, ppb, ppm)"] --> I
    C["Defined HRI Formula\n(same input → same output)"] --> I
    D["Standard Quality Flags\n(VALIDATED / UNVALIDATED / SUSPECT / INVALID)"] --> I
    E["Standard API Shape\nGET /locations/{geohash}/latest"] --> I
    F["Geohash Addressing\n(global, platform-independent)"] --> I
```

Architecture 4 — Source → BXP → Application Workflow

What this represents: The complete workflow for a developer integrating a new sensor or data source with BXP, from first measurement to application display. This workflow is based on the actual working SDK and reference server.

```
flowchart TD
    A["1. Sensor / App captures measurement\n(PM2.5=47.2 µg/m³ at 5.6037°N, 0.187°W)"]
    B["2. Register device with BXP node\nPOST /bxp/v2/devices/register\n→ receive device_token"]
    C["3. Build .bxp.json record\nbxp_sdk.write_bxp() or manually"]
    D["4. SDK computes:\n- geohash from lat/lon\n- BXP_HRI score\n- quality flag\n- SHA-256 payloadHash"]
    E["5. Submit to BXP node\nPOST /bxp/v2/readings\nAuthorization: Bearer device_token"]
    F["6. Node validates:\n- lat/lon range\n- agent values non-negative\n- geohash precision ≥ 5\n- token validity"]
    G["7. Node stores with privacy controls:\n- geohash floor at precision 5 if anonymous\n- SHA-256 hash stored, not plain UUID"]
    H["8. Application queries:\nGET /bxp/v2/locations/s1v0g/latest\nGET /bxp/v2/city/accra"]
     I["9. Application receives:\n{bxpHri: 61.2, bxpHriLevel: 'HIGH',\ncolor: '#CC0000',\nadvice: 'Wear N95 outdoors'}"]
    J["10. Display to user with\nBXP standard color + risk level"]

    A --> B --> C --> D --> E --> F --> G --> H --> I --> J
```

Architecture 5 — Data Lifecycle

What this represents: The complete lifecycle of a BXP data record, from capture through storage, use, and deletion. Cryptographic integrity is maintained throughout.

```
stateDiagram-v2
    [*] --> Captured: Sensor / app measurement

    Captured --> Packaged: SDK builds .bxp.json\n+ payloadHash

    Packaged --> Submitted: POST /bxp/v2/readings

    Submitted --> Validated: Server validates schema,\ncoordinates, agent values
    Submitted --> Rejected: Validation fails → 400

    Validated --> QualityFlagged: QC pipeline assigns\nVALIDATED / UNVALIDATED / SUSPECT

    QualityFlagged --> Stored: INSERT to SQLite\nwith payloadHash

    Stored --> Served: GET /readings/{id}\nGET /locations/{geohash}/latest
    Stored --> Aggregated: GET /locations/{geohash}/aggregate\n(only if k≥5 readings exist)
    Stored --> Verified: GET /readings/{id}/verify\nrecomputes + compares hash

    Verified --> Verified_OK: Hash matches → VERIFIED
    Verified --> Verified_BAD: Hash mismatch → TAMPERED

    Stored --> Deletion_Requested: DELETE /readings/{id}\n(auth required)
    Deletion_Requested --> Deleted: Cryptographic deletion proof\nreturned to requester
    Deleted --> [*]
```

Architecture 6 — Privacy Architecture

What this represents: How BXP protects personal exposure data at every layer. All privacy controls shown here are implemented in the reference server v2.1.

```
graph TD
    subgraph "Data at Source"
        RAW["Raw reading\nExact GPS: 5.6037°N, 0.1870°W\nDevice UUID: 550e8400-..."]
    end

    subgraph "Privacy Controls Applied at Server"
         ANON_LOC["Geohash Floor\nAnonymous submissions:\nprecision truncated to 5\n(~4.9km cell)\nExact location never stored"]
        ANON_ID["Person ID Hashing\nSHA-256(person_id)\nOnly hash stored — never plain ID"]
          K_ANON["k-Anonymity\nAggregate only published\nwhen k≥5 readings exist\nNo individual reading exposed"]
        CRYPTO_DEL["Cryptographic Deletion\nDELETE /readings/{id}\nProof returned\nIrreversible"]
    end

    subgraph "Public Outputs"
        AGG["Aggregate: avg/min/max HRI\nfor geohash cell (k≥5 only)"]
        CITY["City-level data\n(no individual records)"]
    end

    RAW --> ANON_LOC --> AGG
    RAW --> ANON_ID
    RAW --> K_ANON --> AGG
    RAW --> CRYPTO_DEL
    CITY --> AGG
```

Architecture Summary Table

```
Component  |  Scope  |  Status  |  File
FastAPI reference server  |  Node implementation  |   Implemented  |  reference-server/server.py
SQLite persistence  |  Data storage  |   Implemented  |  reference-server/database.py
REST API (20+ endpoints)  |  Protocol interface  |   Implemented  |  reference-server/server.py
BXP_HRI calculator  |  Health risk scoring  |   Implemented  |  sdk/python/bxp_sdk.py
Privacy layer  |  k-anon, geohash floor  |   Implemented  |  reference-server/server.py
Web dashboard  |  Map + charts  |   Implemented  |  reference-server/server.py
Python SDK  |  Developer library  |   Implemented  |  sdk/python/bxp_sdk.py
TypeScript SDK  |  Developer library  |   Implemented  |  sdk/typescript/bxp-sdk.ts
CLI tool  |  Developer tooling  |   Implemented  |  cli/bxp_cli.py
MQTT bridge  |  IoT integration  |   Implemented  |  integrations/mqtt_bridge.py
Docker deployment  |  Infrastructure  |   Implemented  |  Dockerfile, docker-compose.yml
Binary .bxp format  |  Wire format  |   Specified  |  SPEC.md §5.2
BXP volume file system  |  Storage model  |   Specified  |  SPEC.md §4
Federated node sync  |  Decentralisation  |   Specified  |  SPEC.md §8
Arduino/ESP32 SDKs  |  IoT targets  |   Planned  |  v2.1 roadmap
BXP Foundation  |  Governance  |   Proposed  |  SPEC.md §12.1
```
